Gadget 시스템

개요

Gadget은 ZKEVM 회로 구현에서 재사용 가능한 작은 회로 컴포넌트입니다. 각 Gadget은 특정한 검증 로직을 캡슐화하여 여러 회로에서 일관되게 사용할 수 있도록 합니다. `gadgets/` 크레이트는 이러한 재사용 가능한 컴포넌트들의 모음을 제공합니다.

Gadget의 역할

- **재사용성**: 공통 검증 로직을 한 번 구현하여 여러 회로에서 사용
- **모듈성**: 복잡한 회로를 작은 검증 가능한 단위로 분해
- **일관성**: 동일한 검증 로직이 모든 곳에서 동일하게 동작하도록 보장
- **테스트 용이성**: 개별 Gadget을 독립적으로 테스트 가능

주요 Gadget 타입

1. IsZeroGadget

값이 0인지 검증하는 Gadget입니다.

파일: `gadgets/src/is_zero.rs`

동작 원리:

- 주어진 값 `value` 에 대해 `value_inv` 를 증인으로 제공
- `value_inv` 는 `value` 가 0이 아닐 때 `1/value` , 0일 때 0
- `is_zero = 1 - value * value_inv` 표현식으로 0 여부 판단

제약 조건:

```
// value * (1 - value * value_inv) == 0
// 이는 다음을 보장:
// - value == 0이면 is_zero == 1
// - value != 0이면 value_inv == 1/value이고 is_zero == 0
```

진리표:

```
+----+-------+-----------+----------------------+
| ok | value | value_inv | 1 - value · value_inv |
+----+-------+-----------+----------------------+
| V | 0 | 0 | 1 |
| V | x | 1/x | 0 |
+----+-------+-----------+----------------------+
```

사용 예시:

```
let  is_zero_config = IsZeroChip::configure(
meta,
|meta| meta.query_selector(q_enable),
|meta| meta.query_advice(value_column, Rotation::cur()),
value_inv_column,
);
```

```
// 할당 시
chip.assign(region, offset, Value::known(value))?;
```

2. IsEqualGadget

두 값이 같은지 검증하는 Gadget입니다.

파일: `gadgets/src/is_equal.rs`

동작 원리:

- 내부적으로 `IsZeroChip` 을 사용
- `lhs - rhs` 가 0인지 확인하여 동등성 검증
- `is_equal = is_zero(lhs - rhs)`

구성:

```
pub struct IsEqualConfig<F> {
is_zero_config: IsZeroConfig<F>,
pub is_equal_expression: Expression<F>,
}
```

사용 예시:

```
let is_equal_config = IsEqualChip::configure(
meta,
|meta| meta.query_selector(q_enable),
|meta| meta.query_advice(lhs_column, Rotation::cur()),
```

```
|meta| meta.query_advice(rhs_column, Rotation::cur()),
);


// 할당 시
chip.assign(region, offset, Value::known(lhs), Value::known(rhs))?;
```

3. LtGadget (Less Than)

두 값의 대소 관계를 검증하는 Gadget입니다.

파일: `gadgets/src/less_than.rs`

동작 원리:

- `lhs < rhs` 를 검증
- `diff = lhs - rhs + (lt ? range : 0)` 를 바이트로 분해
- 각 바이트가 u8 범위 내에 있는지 룩업 테이블로 검증

제약 조건:

```
// check_a: lhs - rhs - diff_bytes + (lt * range) == 0
// check_b: lt * (1 - lt) == 0 (lt는 boolean)
// 각 diff_byte는 u8_table에서 룩업
```

구성:

```
pub struct LtConfig<F, const N_BYTES: usize> {
pub lt: Column<Advice>, // 결과 (0 또는 1)
pub diff: [Column<Advice>; N_BYTES], // 차이의 바이트 표현
```

```
pub u8_table: TableColumn, // 범위 검증용 테이블
pub range: F, // 2^(N_BYTES * 8)
}
```

사용 예시:

```
// 8바이트 (64비트) 비교
let lt_config = LtChip::<F, 8>::configure(
meta,
|meta| meta.query_selector(q_enable),
|meta| meta.query_advice(lhs_column, Rotation::cur()),
|meta| meta.query_advice(rhs_column, Rotation::cur()),
u8_table,
);

// 할당 시
chip.assign(region, offset, lhs, rhs)?;
```

4. BinaryNumberGadget

값을 이진 비트로 표현하고 특정 값과 비교하는 Gadget입니다.

파일: `gadgets/src/binary_number.rs`

동작 원리:

- N비트 이진 표현으로 값을 분해

- 각 비트가 0 또는 1인지 제약
- 특정 값과의 동등성을 비트 단위로 검증

제약 조건:

```
// 각 비트에 대해: bit * (1 - bit) == 0
// 값 검증: value == sum(bit[i] * 2^i)
// 유효하지 않은 비트 패턴 금지
```

구성:

```
pub struct BinaryNumberConfig<T, const N: usize> {
pub bits: [Column<Advice>; N],
_marker: PhantomData<T>,
}
```

주요 메서드:

```
// 비트에서 값 표현식 생성
pub fn value<F: Field>(&self, rotation: Rotation)
-> impl FnOnce(&mut VirtualCells<'_, F>) -> Expression<F>

// 특정 값과의 동등성 표현식
pub fn value_equals<F: Field, S: AsBits<N>>(
&self, value: S, rotation: Rotation
```

```
) -> impl FnOnce(&mut VirtualCells<'_, F>) -> Expression<F>
```

5. ComparatorGadget

LT와 EQ를 동시에 제공하는 복합 Gadget입니다.

파일: `gadgets/src/comparator.rs`

동작 원리:

- `LtChip` 과 `IsEqualChip` 을 결합
- 한 번의 할당으로 `(lt, eq)` 결과 제공
- GT는 `!lt && !eq` 로 간접적으로 계산 가능

구성:

```
pub struct ComparatorConfig<F, const N_BYTES: usize> {
pub lt_chip: LtChip<F, N_BYTES>,
pub eq_chip: IsEqualChip<F>,
}
```

사용 예시:

```
let comparator_config = ComparatorChip::<F, 8>::configure(
meta,
|meta| meta.query_selector(q_enable),
|meta| meta.query_advice(lhs_column, Rotation::cur()),
|meta| meta.query_advice(rhs_column, Rotation::cur()),
u8_table,
```

```
);


// 표현식 얻기
let (lt_expr, eq_expr) = config.expr(meta);
```

6. BatchedIsZeroGadget

여러 값이 모두 0인지 한 번에 검증하는 Gadget입니다.

파일: `gadgets/src/batched_is_zero.rs`

동작 원리:

- N개의 값이 모두 0인지 검증
- `nonempty_witness` : 0이 아닌 값이 있으면 그 역수, 없으면 0
- `is_zero` : 모든 값이 0이면 1, 아니면 0

제약 조건:

```
// is_zero는 boolean: is_zero * (is_zero - 1) == 0
// 0이 아닌 값이 있으면 is_zero는 0: is_zero * value[i] == 0 (모든 i)
// 모든 값이 0이면 is_zero는 1:
// (1 - is_zero) * product(1 - value[i] * nonempty_witness) == 0
```

구성:

```
pub struct BatchedIsZeroConfig {
pub is_zero: Column<Advice>,
```

```
pub  nonempty_witness: Column<Advice>,
}
```

사용 예시:

```
// 3개 값 검증
let  config = BatchedIsZeroChip::<F, 3>::configure(
meta,
(FirstPhase, FirstPhase),
|meta| meta.query_selector(q_enable),
|meta| [
meta.query_advice(col0, Rotation::cur()),
meta.query_advice(col1, Rotation::cur()),
meta.query_advice(col2, Rotation::cur()),
],
);


// 할당 시
chip.assign(region, offset, Value::known([val0, val1, val2]))?;
```

7. MulAddGadget

`a * b + c == d (mod 2^256)` 검증을 위한 Gadget입니다.

파일: `gadgets/src/mul_add.rs`

동작 원리:

- 256비트 곱셈과 덧셈을 64비트 limb로 분해
- 중간 carry 값들을 16비트 단위로 분해하여 범위 검증
- 8행에 걸쳐 레이아웃 구성

회로 레이아웃:

```
| q_step | col0 | col1 | col2 | col3 |
|--------|-----------|-----------|-----------|-----------|
| 1 | a_limb0 | a_limb1 | a_limb2 | a_limb3 |
| 0 | b_limb0 | b_limb1 | b_limb2 | b_limb3 |
| 0 | c_lo | c_hi | d_lo | d_hi |
| 0 | carry_lo0 | carry_lo1 | carry_lo2 | carry_lo3 |
| 0 | carry_lo4 | - | - | - |
| 0 | carry_hi0 | carry_hi1 | carry_hi2 | carry_hi3 |
| 0 | carry_hi4 | - | - | - |
| 0 | - | - | - | - |
```

제약 조건:

```
// t0 = a[0] * b[0]
// t1 = a[0] * b[1] + a[1] * b[0]
// t2 = a[0] * b[2] + a[1] * b[1] + a[2] * b[0]
// t3 = a[0] * b[3] + a[1] * b[2] + a[2] * b[1] + a[3] * b[0]

// check_a: t0 + t1 * 2^64 + c_lo == d_lo + carry_lo * 2^128
```

```
// check_b: t2 + t3 * 2^64 + c_hi + carry_lo == d_hi + carry_hi * 2^128
```

구성:

```rust
pub struct MulAddConfig<F> {
pub col0: Column<Advice>,
pub col1: Column<Advice>,
pub col2: Column<Advice>,
pub col3: Column<Advice>,
pub overflow: Expression<F>,
pub u16_table: TableColumn,
pub range_check_64: UIntRangeCheckChip<F, 8>,
pub range_check_128: UIntRangeCheckChip<F, 4>,
}
```

Gadget 구현 패턴

1. Config-Chip 분리 패턴

모든 Gadget은 Config와 Chip으로 분리됩니다:

```rust
// Config: 회로 구조 정의
#[derive(Clone, Debug)]
pub struct GadgetConfig<F> {
pub columns: Vec<Column<Advice>>,
```

```
pub  expression: Expression<F>,
}


// Chip: Config를 사용하는 래퍼
#[derive(Clone, Debug)]
pub  struct GadgetChip<F> {
config: GadgetConfig<F>,
}
```

장점:

- Config는 회로 설정 단계에서 생성
- Chip은 증인 할당 단계에서 사용
- 동일한 Config로 여러 Chip 인스턴스 생성 가능

2. Instruction 트레이트 패턴

각 Gadget은 `Instruction` 트레이트를 구현합니다:

```
pub  trait GadgetInstruction<F: Field> {
fn  assign(
&self,
region: &mut Region<'_, F>,
offset: usize,
// ... 입력 파라미터
) -> Result<(), Error>;
```

```
}
```

장점:

- 일관된 인터페이스 제공
- 다형성을 통한 유연한 사용
- 테스트 및 모킹 용이

3. Configure 메서드 패턴

모든 Gadget은 정적 `configure` 메서드를 제공합니다:

```
impl<F: Field> GadgetChip<F> {
pub fn configure(
meta: &mut ConstraintSystem<F>,
q_enable: impl FnOnce(&mut VirtualCells<'_, F>) -> Expression<F>,
// ... 입력 표현식 생성자
) -> GadgetConfig<F> {
// 제약 조건 생성
meta.create_gate("gadget gate", |meta| {
let q_enable = q_enable(meta);
// ... 제약 조건
});
GadgetConfig { /* ... */ }
}
pub fn construct(config: GadgetConfig<F>) -> Self {
```

```
Self { config }
}
}
```

특징:

- `q_enable` : 선택자 표현식 (이 Gadget이 활성화되는 행)
- 입력 표현식: 클로저로 제공하여 유연성 확보
- 반환된 Config는 나중에 Chip 생성에 사용

4. Expression 반환 패턴

많은 Gadget은 결과를 Expression으로 제공합니다:

```
impl<F: Field> GadgetConfig<F> {
pub fn expr(&self) -> Expression<F> {
self.result_expression.clone()
}
pub fn expr_at(
&self,
meta: &mut VirtualCells<'_, F>,
at: Rotation,
// ... 입력
) -> Expression<F> {
// 특정 rotation에서의 표현식
}
```

```
}
```

장점:

- Gadget 결과를 다른 제약 조건에서 직접 사용 가능
- 추가 컬럼 할당 없이 논리 조합 가능

유틸리티 함수

`gadgets/src/util.rs` 는 표현식 조작을 위한 유틸리티를 제공합니다:

논리 연산

```
// AND: 모든 입력이 1일 때만 1
pub mod and {
pub fn expr<F: Field, E: Expr<F>, I: IntoIterator<Item = E>>(
inputs: I
) -> Expression<F>
}

// OR: 하나라도 1이면 1
pub mod or {
pub fn expr<F: Field, E: Expr<F>, I: IntoIterator<Item = E>>(
inputs: I
) -> Expression<F>
}
```

```
// NOT: 0이면 1, 1이면 0
pub mod not {
pub fn expr<F: Field, E: Expr<F>>(b: E) -> Expression<F>
}

// XOR: 정확히 하나만 1일 때 1
pub mod xor {
pub fn expr<F: Field, E: Expr<F>>(a: E, b: E) -> Expression<F>
}
```

선택 연산

```
// SELECT: selector가 1이면 when_true, 0이면 when_false
pub mod select {
pub fn expr<F: Field>(
selector: Expression<F>,
when_true: Expression<F>,
when_false: Expression<F>,
) -> Expression<F>
}
```

합계 연산

```
// SUM: 모든 입력의 합
pub mod sum {
pub fn expr<F: Field, E: Expr<F>, I: IntoIterator<Item = E>>(
inputs: I
) -> Expression<F>
}
```

바이트 변환

```
// 바이트 배열을 단일 표현식으로 변환
pub fn expr_from_bytes<F: Field, E: Expr<F>>(bytes: &[E]) -> Expression<F> {
let mut value = 0.expr();
let mut multiplier = F::ONE;
for byte in bytes.iter() {
value = value + byte.expr() * multiplier;
multiplier *= F::from(256);
}
value
}
```

재사용성 확보 전략

1. 제네릭 파라미터 활용

많은 Gadget은 제네릭 파라미터를 사용하여 유연성을 제공합니다:

```
// N_BYTES: 비교할 바이트 수
pub struct LtChip<F, const N_BYTES: usize> {
config: LtConfig<F, N_BYTES>,
}

// 사용 예시
LtChip::<F, 8>::configure(...) // 64비트 비교
LtChip::<F, 32>::configure(...) // 256비트 비교
```

2. 클로저 기반 입력

입력을 클로저로 받아 다양한 소스에서 사용 가능:

```
pub fn configure(
meta: &mut ConstraintSystem<F>,
q_enable: impl FnOnce(&mut VirtualCells<'_, F>) -> Expression<F>,
value: impl FnOnce(&mut VirtualCells<'_, F>) -> Expression<F>,
// ...
) -> Config<F>
```

장점:

- 고정 컬럼, advice 컬럼, 복잡한 표현식 모두 지원
- 다른 rotation에서의 값 참조 가능

- 여러 컬럼의 조합 표현식 사용 가능

3. 표현식 기반 결과

결과를 Expression으로 반환하여 조합 가능:

```
// IsZero 결과를 다른 제약에서 사용
let is_zero_expr = is_zero_config.expr();
meta.create_gate("custom gate", |meta| {
vec![q_enable * is_zero_expr * some_other_expr]
});
```

4. 컴포지션 패턴

복잡한 Gadget은 단순한 Gadget을 조합하여 구성:

```
// IsEqualChip은 IsZeroChip을 내부적으로 사용
pub struct IsEqualConfig<F> {
is_zero_config: IsZeroConfig<F>,
pub is_equal_expression: Expression<F>,
}

// ComparatorChip은 LtChip과 IsEqualChip을 조합
pub struct ComparatorConfig<F, const N_BYTES: usize> {
pub lt_chip: LtChip<F, N_BYTES>,
pub eq_chip: IsEqualChip<F>,
```

```
}
```

테스트 전략

모든 Gadget은 포괄적인 테스트를 포함합니다:

1. 단위 테스트

각 Gadget은 독립적인 테스트 회로를 가집니다:

```
#[test]
fn  test_gadget() {
#[derive(Clone, Debug)]
struct TestCircuitConfig<F> {
q_enable: Selector,
// ... 테스트용 컬럼
gadget: GadgetConfig<F>,
}
#[derive(Default)]
struct TestCircuit<F: Field> {
values: Option<Vec<TestValue>>,
_marker: PhantomData<F>,
}
impl<F: Field> Circuit<F> for TestCircuit<F> {
// ... 구현
```

```
}

// 정상 케이스

let circuit = TestCircuit { values: Some(valid_values), ... };

let prover = MockProver::run(k, &circuit, vec![]).unwrap();

prover.assert_satisfied();

// 오류 케이스

let circuit = TestCircuit { values: Some(invalid_values), ... };

let prover = MockProver::run(k, &circuit, vec![]).unwrap();

assert!(prover.verify().is_err());

}
```

2. 경계 조건 테스트

각 Gadget은 경계 조건을 테스트합니다:

```
#[test]

fn test_boundary_conditions() {

// 0 값

test_circuit(0, expected_result);

// 최대 값

test_circuit(u64::MAX, expected_result);

// 경계 근처 값

test_circuit(1, expected_result);

test_circuit(u64::MAX - 1, expected_result);
```

```
}
```

3. 랜덤 테스트

많은 Gadget은 랜덤 입력으로 테스트합니다:

```
#[test]
fn test_random_values() {
let mut rng = rand::thread_rng();
for _ in 0..100 {
let value = rng.gen::<u64>();
test_circuit(value, compute_expected(value));
}
}
```

실제 사용 예시

EVM Circuit에서의 IsZero 사용

```
// EVM Circuit의 ISZERO opcode 구현
pub struct IsZeroGadget<F> {
is_zero: IsZeroConfig<F>,
// ...
}
```

```rust
impl<F: Field> ExecutionGadget<F> for IsZeroGadget<F> {
fn  configure(cb: &mut ConstraintBuilder<F>) -> Self {
let  value = cb.query_word_rlc();
let  is_zero = IsZeroChip::configure(
cb.base.meta,
|meta| cb.query_selector(),
|meta| value.expr(),
cb.base.meta.advice_column(),
);
// 결과를 스택에 푸시
cb.stack_push(is_zero.expr());
Self { is_zero, /* ... */ }
}
}
```

State Circuit에서의 Comparator 사용

```rust
// RW 테이블 정렬 검증
let  comparator = ComparatorChip::configure(
meta,
|meta| meta.query_selector(q_enable),
|meta| current_key_expr(meta),
```

```
|meta| next_key_expr(meta),
u8_table,
);


// current_key <= next_key 검증
meta.create_gate("lexicographic ordering", |meta| {
let (lt, eq) = comparator.expr(meta);
vec![q_enable * (1.expr() - lt - eq)] // !(lt || eq) == 0
});
```

기타 유용한 Gadget

MonotoneGadget

값이 단조 증가하는지 검증하는 Gadget입니다.

파일: `gadgets/src/monotone.rs`

용도:

- 카운터가 항상 증가하는지 검증
- 정렬된 시퀀스 검증

RangeGadget

값이 특정 범위 내에 있는지 검증하는 Gadget입니다.

파일: `gadgets/src/range.rs`

용도:

- 값이 특정 비트 수로 표현 가능한지 검증
- 오버플로우 방지

EvmWordGadget

EVM의 256비트 워드를 다루는 Gadget입니다.

파일: `gadgets/src/evm_word.rs`

용도:

- 256비트 값을 128비트 limb로 분해
- RLC (Random Linear Combination) 계산
- 워드 간 산술 연산

Gadget 설계 원칙

1. 최소 제약 조건

Gadget은 필요한 최소한의 제약 조건만 포함해야 합니다:

```
// 좋은 예: IsZero는 단 하나의 제약 조건만 사용
meta.create_gate("is_zero gate", |meta| {
[q_enable * value * (1.expr() - value * value_inv)]
});

// 나쁜 예: 불필요한 중복 제약
```

2. 명확한 인터페이스

Gadget의 입력과 출력은 명확해야 합니다:

```
// 좋은 예: 명확한 파라미터 이름과 타입
pub fn configure(
meta: &mut ConstraintSystem<F>,
q_enable: impl FnOnce(&mut VirtualCells<'_, F>) -> Expression<F>,
lhs: impl FnOnce(&mut VirtualCells<'_, F>) -> Expression<F>,
rhs: impl FnOnce(&mut VirtualCells<'_, F>) -> Expression<F>,
) -> Config<F>

// 나쁜 예: 모호한 파라미터
pub fn configure(
meta: &mut ConstraintSystem<F>,
f1: impl Fn(...) -> Expression<F>,
f2: impl Fn(...) -> Expression<F>,
) -> Config<F>
```

3. 독립성

Gadget은 가능한 한 독립적이어야 합니다:

- 외부 상태에 의존하지 않음
- 다른 Gadget과의 결합도 최소화
- 필요한 경우에만 다른 Gadget 조합

4. 테스트 가능성

모든 Gadget은 독립적으로 테스트 가능해야 합니다:

```
#[cfg(test)]
mod test {
// 단위 테스트
#[test]
fn test_basic_functionality() { /* ... */ }
// 경계 조건 테스트
#[test]
fn test_edge_cases() { /* ... */ }
// 오류 케이스 테스트
#[test]
fn test_error_cases() { /* ... */ }
}
```

성능 고려사항

1. 컬럼 사용 최소화

각 Gadget은 필요한 최소한의 컬럼만 사용해야 합니다:

```
// IsZero는 단 하나의 advice 컬럼만 사용
pub struct IsZeroConfig<F> {
pub value_inv: Column<Advice>,
pub is_zero_expression: Expression<F>,
```

```
}
```

2. 룩업 테이블 재사용

여러 Gadget이 동일한 룩업 테이블을 공유할 수 있습니다:

```
// u8 테이블을 여러 Gadget에서 공유
let u8_table = meta.lookup_table_column();

let lt_config = LtChip::configure(meta, ..., u8_table);
let mul_add_config = MulAddChip::configure(meta, ..., u8_table);
```

3. 표현식 복잡도 관리

복잡한 표현식은 중간 컬럼을 사용하여 분해:

```
// 복잡한 표현식을 여러 단계로 분해
let intermediate = meta.advice_column();

meta.create_gate("step 1", |meta| {
let result = complex_expr_part1(meta);
vec![q_enable * (meta.query_advice(intermediate, Rotation::cur()) - result)]
});

meta.create_gate("step 2", |meta| {
```

```
let input = meta.query_advice(intermediate, Rotation::cur());

let result = complex_expr_part2(meta, input);

// ...

});
```

요약

Gadget 시스템은 ZKEVM 구현의 핵심 구성 요소입니다:

1. **재사용성**: 공통 검증 로직을 한 번 구현하여 여러 곳에서 사용
2. **모듈성**: 복잡한 회로를 작은 검증 가능한 단위로 분해
3. **일관성**: 동일한 검증 로직이 모든 곳에서 동일하게 동작
4. **테스트 용이성**: 개별 Gadget을 독립적으로 테스트 가능

주요 Gadget 타입:

- **IsZeroGadget**: 0 검증
- **IsEqualGadget**: 동등성 검증
- **LtGadget**: 대소 비교
- **BinaryNumberGadget**: 이진 표현
- **ComparatorGadget**: 복합 비교
- **BatchedIsZeroGadget**: 다중 0 검증
- **MulAddGadget**: 곱셈-덧셈 검증

구현 패턴:

- Config-Chip 분리
- Instruction 트레이트

- Configure 메서드
- Expression 반환
- 클로저 기반 입력

이러한 Gadget들은 EVM Circuit, State Circuit, Bytecode Circuit 등 모든 주요 회로에서 광범위하게 사용되며, 코드 재사용성과 검증 정확성을 크게 향상시킵니다.

관련 문서

- Halo2 통합 - Halo2 API와 Gadget의 관계
- 제약 조건 구축 - Gadget에서 제약 조건 생성 방법
- EVM Circuit - Gadget 사용 예시
- State Circuit - Comparator 사용 예시